

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY (INCIDENT INVESTIGATION, ROOT CAUSE ANALYSIS AND RISK ASSESSMENT)

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO5 – Precision (BL2, BL3, BL4)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Conduct Incident Investigation, Root Cause Analysis and Risk Assessment for Application Security Incidents.		
Student Name:	Adhithyan S	Reg. No.:	192421102
Section / Batch:		Date:	

Instructions to Students

- Investigate a real-world / simulated application security incident from detection through resolution.
- Document the complete incident timeline, evidence collected and containment actions taken.
- Perform a structured Root Cause Analysis using techniques such as the 5-Whys and Fishbone (Ishikawa) diagram.
- Carry out a Risk Assessment covering likelihood, impact, and a risk matrix for related and residual risks.
- Recommend corrective and preventive actions (CAPA) and long-term security control improvements.
- Use clear diagrams, tables and professionally formatted sections throughout the report.

Question Type	Focus Area	Questions	Marks
CASE STUDY	Incident Investigation, Root Cause Analysis & Risk Assessment	Q1	Q1 – 100
GRAND TOTAL:		100 Marks	

CASE STUDY REPORT

Incident Investigation, Root Cause Analysis and Risk Assessment of a CI/CD Secrets-Leak Security Incident

1. Executive Summary

This report presents a structured case study of an application security incident investigated within a DevSecOps environment. The incident involved the accidental exposure of a cloud provider access key (AWS Identity and Access Management credential) inside a public source-code repository connected to the organisation's Continuous Integration / Continuous Deployment (CI/CD) pipeline. The report walks through incident detection, containment, evidence collection, root cause analysis using the 5-Whys technique and a Fishbone (Ishikawa) diagram, a formal risk assessment using a likelihood-impact matrix, and the corrective and preventive actions recommended to reduce recurrence. The objective is to demonstrate a complete, auditable incident-response and risk-management methodology consistent with DevSecOps and application-security best practice.

2. Introduction

Modern software delivery pipelines integrate development, security and operations activities into a single automated workflow. While this accelerates delivery, it also expands the attack surface: a single misconfigured pipeline step, an exposed secret, or an unreviewed commit can lead to a serious security incident. Incident investigation and root cause analysis (RCA) are core DevSecOps disciplines used to understand not just what happened during an incident, but why it happened, so that the underlying weaknesses in people, process and technology can be permanently corrected rather than only patched superficially. Risk assessment complements this by quantifying the likelihood and impact of related risks so that remediation effort can be prioritised objectively.

3. Incident Overview

On the date of occurrence, a developer working on a feature branch of the organisation's web-application repository committed a configuration file that inadvertently contained a live AWS access key and secret, intended only for local testing. The repository was hosted on a publicly accessible Git hosting service. Within approximately two hours, automated credential-scanning bots operated by an external threat actor identified the exposed key and began issuing unauthorised API calls against the organisation's cloud account, attempting to enumerate storage buckets and spin up compute resources for cryptomining.

3.1 Incident Classification

Attribute	Details
Incident ID	INC-2026-0091
Category	Credential Exposure / Unauthorised Cloud Access
Severity	High (Confidentiality & Integrity impact)
Affected Asset	CI/CD pipeline, Cloud IAM key, associated cloud resources
Detection Source	Cloud provider anomaly alert (CloudTrail / GuardDuty equivalent)
Reported By	Security Operations Centre (SOC) on-call analyst

4. Incident Investigation Process

The investigation followed a structured six-stage workflow, moving from detection through to documentation and continuous improvement, as illustrated in Figure 1. Each stage produced artefacts (alerts, logs, screenshots, interview notes) that were preserved under chain-of-custody for the final report.

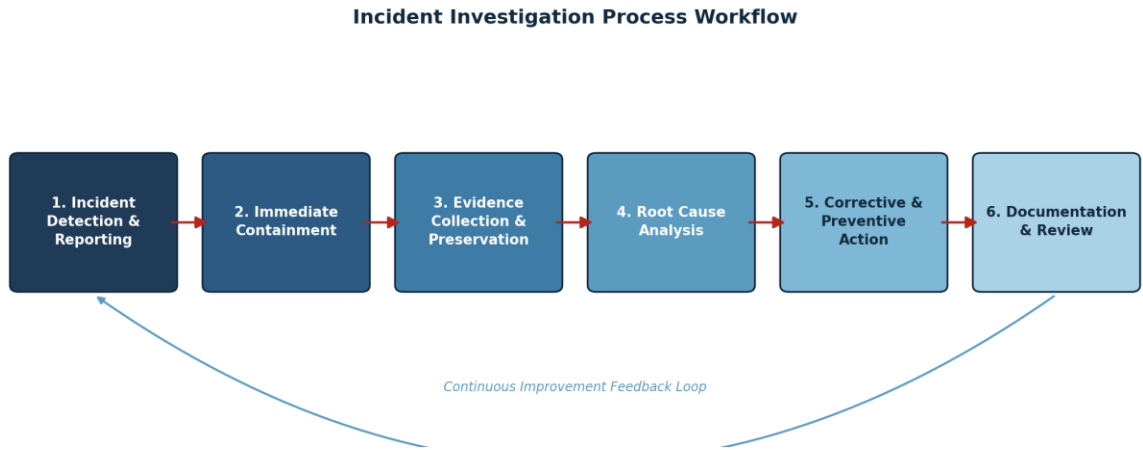


Figure 1: Incident Investigation Process Workflow

4.1 Detection and Containment

The cloud provider's anomaly-detection service flagged a burst of API calls from an unfamiliar IP address range approximately two hours after the credential was committed. The on-call analyst validated the alert, confirmed unauthorised activity, and immediately revoked the exposed IAM key, rotated all related secrets, and isolated the affected CI/CD runner from the network to halt further damage.

4.2 Evidence Collection and Preservation

Investigators collected: (a) the Git commit history and diff showing the exact commit that introduced the secret; (b) CI/CD pipeline execution logs; (c) cloud provider audit logs (API call history, source IPs, timestamps); (d) IAM key usage history; and (e) interview notes from the developer who authored the commit. All evidence was hashed and stored in a write-once evidence repository to preserve integrity for the post-incident review.

4.3 Incident Timeline

The complete sequence of events, from initial compromise to closure, is summarised in the timeline below.

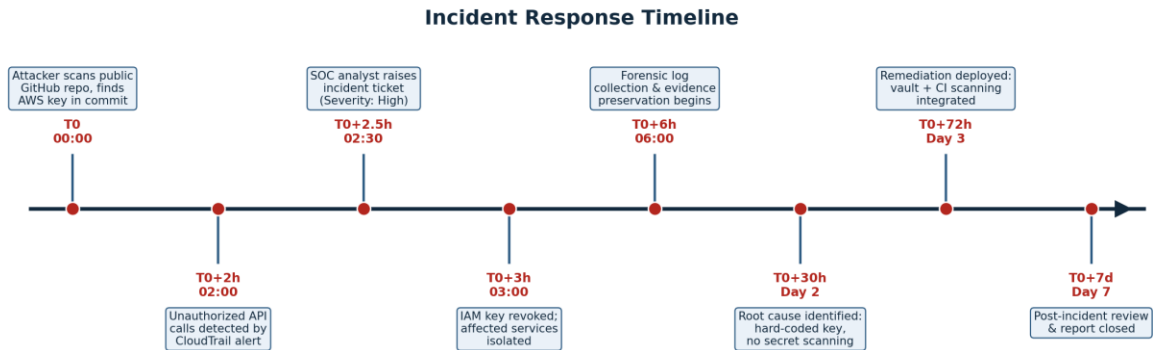


Figure 2: Incident Response Timeline

5. Root Cause Analysis

Root cause analysis was performed using two complementary techniques: the 5-Whys method to trace the causal chain to its origin, and a Fishbone (Ishikawa) diagram to organise contributing factors across People, Process, Technology and Environment categories.

5.1 The 5-Whys Analysis

Step	Question	Answer
Why 1	Why were unauthorised cloud API calls made?	Because a valid AWS access key was used by an external party.
Why 2	Why did an external party have a valid key?	Because the key was committed in plaintext to a public repository.
Why 3	Why was the key committed to the repository?	Because the developer hard-coded it locally for quick testing and forgot to remove it.
Why 4	Why was this not caught before merge?	Because no automated secret-scanning or pre-commit hook existed in the pipeline.
Why 5	Why was there no secret-scanning control?	Because the DevSecOps toolchain had not yet integrated a secrets-detection gate into CI, and security requirements were not enforced at the process level.

The 5-Whys analysis shows that the immediate trigger (a hard-coded key) was enabled by a deeper systemic gap: the absence of automated preventive controls in the CI/CD pipeline and insufficient enforcement of secure-coding policy.

5.2 Fishbone (Ishikawa) Diagram

To ensure no contributing factor was overlooked, causes were also organised across four standard categories, as shown in Figure 3.

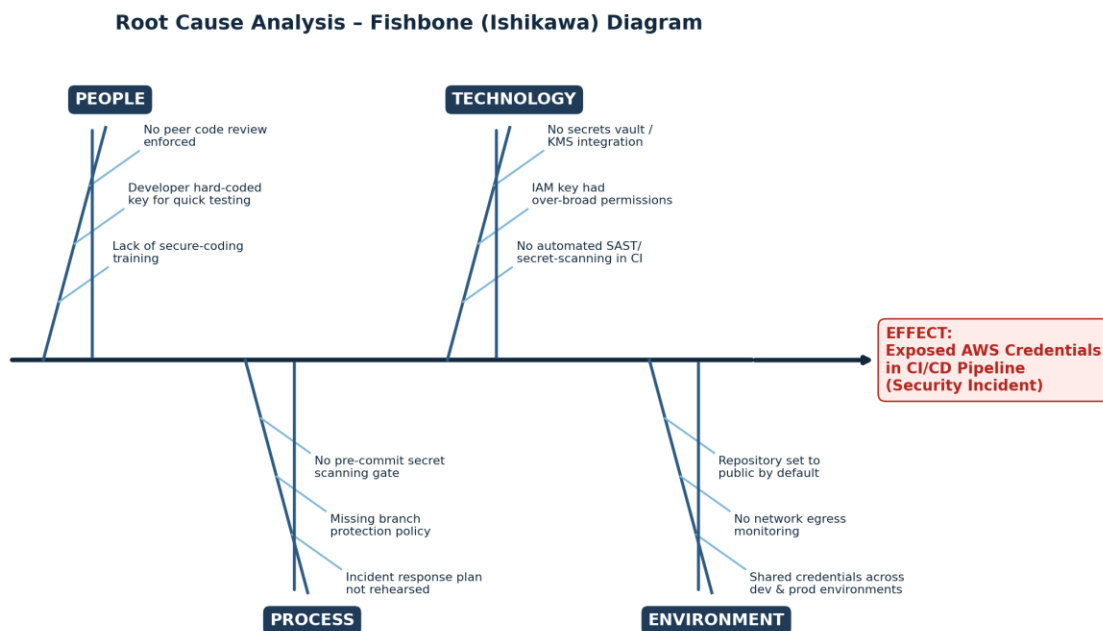


Figure 3: Fishbone (Ishikawa) Diagram – Root Cause Categories

The analysis confirms that the incident was multi-causal: a human error (hard-coding a key) combined with absent technical controls (no secret scanning, no vault integration), weak process governance (no branch protection, untested incident-response plan) and a permissive environment (public repository default, shared dev/prod credentials) collectively created the conditions for the breach.

6. Risk Assessment

Following root cause identification, a formal risk assessment was conducted to evaluate the residual and related risks stemming from this incident, using a standard 5×5 likelihood-impact matrix (Figure 4). Each identified risk was scored as Likelihood × Impact, with higher scores indicating higher priority for mitigation.

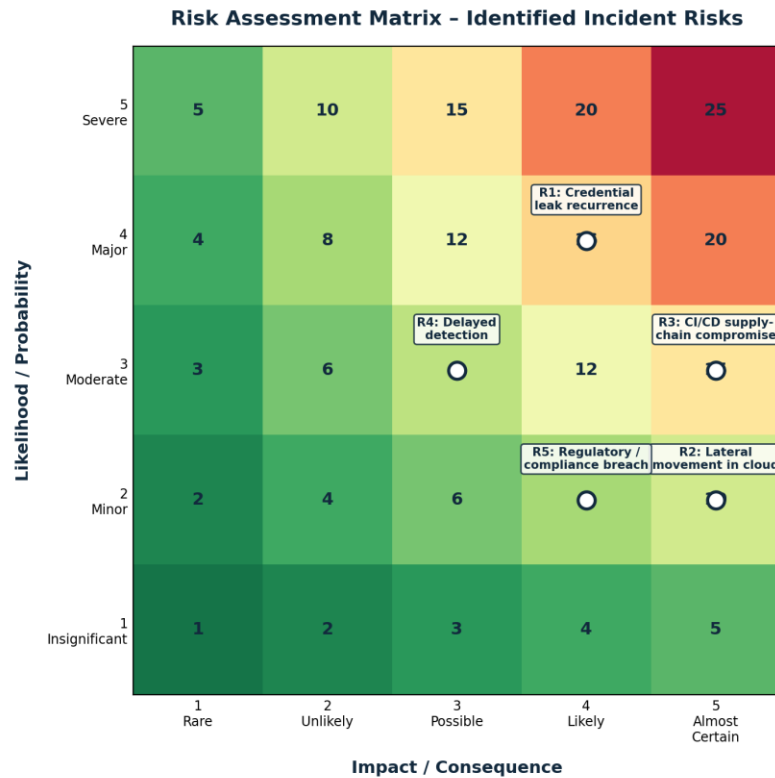


Figure 4: Risk Assessment Matrix – Identified Incident Risks

Risk ID	Description	Likelihood	Impact	Risk Score	Priority
R1	Recurrence of credential leak via another hard-coded secret	4 – Likely	4 – Major	16	High
R2	Lateral movement within cloud account using compromised key	2 – Unlikely	5 – Severe	10	Medium
R3	CI/CD supply-chain compromise via injected malicious dependency	3 – Possible	5 – Severe	15	High
R4	Delayed detection of future incidents due to alert fatigue	3 – Possible	3 – Moderate	9	Medium
R5	Regulatory / compliance breach from cloud data exposure	2 – Unlikely	4 – Major	8	Medium

Risks R1 and R3 fall into the High priority band and require immediate mitigation, while R2, R4 and R5 are Medium priority and should be addressed within the next remediation cycle.

7. Corrective and Preventive Actions (CAPA)

7.1 Immediate Corrective Actions

- Revoke and rotate the exposed IAM key and all credentials shared with the same repository.
- Purge the secret from Git history using history-rewriting tools and force-push a cleaned repository.
- Review CloudTrail logs for all actions performed using the compromised key and reverse any unauthorised changes.

7.2 Preventive Actions

- Integrate automated secret-scanning (e.g., pre-commit hooks and CI-stage scanners) to block commits containing credentials.
- Adopt a centralised secrets vault / KMS so that no credentials are ever stored in source code or configuration files.
- Enforce branch-protection rules requiring mandatory peer code review before merge to protected branches.
- Apply least-privilege IAM policies, scoping keys to only the permissions required for a specific task.
- Set all new repositories to private by default and enforce this via organisation-level policy.
- Conduct periodic secure-coding training and run tabletop incident-response exercises at least twice a year.

7.3 Detective / Monitoring Improvements

- Enable real-time anomaly alerts for unusual API call patterns and geographic access anomalies.
- Implement network egress monitoring to flag unexpected outbound connections from CI/CD runners.

8. Lessons Learned

The incident demonstrated that a single unreviewed commit can bypass otherwise mature security intentions when automated preventive controls are missing. It reinforced the core DevSecOps principle that security must be embedded as code (secret scanning, policy-as-code, least privilege) rather than relying solely on developer discipline. The post-incident review also highlighted the value of a rehearsed incident-response plan: containment was achieved quickly once detected, but detection itself was delayed because monitoring depended on a single alerting source.

9. Conclusion

This case study illustrates a complete DevSecOps incident-management lifecycle — from detection and containment through structured root cause analysis and formal risk assessment, to actionable corrective and preventive measures. Applying the 5-Whys and Fishbone techniques together provided both a linear causal trace and a holistic view across People, Process, Technology and Environment, while the risk matrix ensured that follow-up mitigation work is prioritised by objective risk scoring. Implementing the recommended CAPA items — particularly automated secret scanning and least-privilege IAM design — would significantly reduce the likelihood of a similar incident recurring.

10. References

1. NIST SP 800-61 Rev. 2, Computer Security Incident Handling Guide, National Institute of Standards and Technology.
2. OWASP DevSecOps Guideline, OWASP Foundation.
3. ISO/IEC 27005:2022, Information Security Risk Management, International Organization for Standardization.
4. Cloud Security Alliance, Top Threats to Cloud Computing Reports.
5. Ishikawa, K., Guide to Quality Control, Asian Productivity Organization.